

Laboratório de Programação Orientada a Objetos I

Polimorfismo e fluxos

REALIZAR APÓS Módulo 4 / Semana 4

Módulos 3 a 4 | 120 minutos | 100 pontos

Curso principal: Laboratório de Programação Orientada a Objetos - Parte 1 | <https://pt.coursera.org/learn/lab-poo-parte-1>

Conteúdo verificável: Tipagem; coleções; interfaces; classes abstratas; polimorfismo; exceções; I/O e network streams

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Polimorfismo em uma coleção

Uma coleção reúne Circulo e Retangulo, ambos com `area()`. Descreva uma interface ou classe abstrata `Forma` e escreva o cálculo da área total sem testar o tipo concreto. Use círculo de raio 1 e retângulo 2×3 como exemplo. Explique a vantagem sobre uma sequência de ifs por tipo.

Desenvolvimento / justificativa / esquema:

Tipagem e substituição

Um método aceita uma coleção de objetos que implementam `imprimir()`. Compare a verificação estática em Java com a verificação em execução em Python. Uma subclasse substitui `imprimir()` por um método que sempre lança erro. Por que a relação sintática de herança não garante substituição correta? Proponha um teste do contrato.

Desenvolvimento / justificativa / esquema:

Arquivos e exceções

Implemente em Python uma função que leia inteiros de um arquivo, um por linha, e devolva sua soma. Linhas em branco são ignoradas; uma linha inválida deve produzir uma mensagem com o número da linha, e arquivo inexistente deve ser distinguido. Garanta fechamento mesmo quando há erro.

Desenvolvimento / justificativa / esquema:

Fluxos e limites de mensagens

Um cliente envia as linhas 'SOMA 2 3

' e 'SOMA 4 5

' por um fluxo de rede. Uma leitura devolve 'SOMA 2', a próxima ' 3

SOMA 4 5

'. Descreva um receptor correto usando buffer e delimitador. Escreva pseudocódigo, trate fechamento com linha incompleta e separe transporte, análise e cálculo em responsabilidades distintas.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Polimorfismo em uma coleção (15 pontos)

O contrato comum é `area()` retornar área não negativa. Em Python:

```
def area_total(formas):  
    return sum(f.area() for f in formas)
```

Em Java uma interface `Forma` com `double area()` e uma `List<Forma>` fornecem o mesmo despacho. A soma é $\pi+6$. Acrescentar uma nova forma exige implementar seu contrato, sem editar a função agregadora. Classe abstrata pode compartilhar estado e implementação; interface expressa principalmente o contrato. Aceitar exemplos concretos equivalentes.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Tipagem e substituição (25 pontos)

Java verifica compatibilidade de tipos declarados antes da execução, embora falhas de execução ainda ocorram. Python permite objetos distintos que ofereçam o comportamento necessário; ausência do método pode falhar em execução. A subclasse rompe a expectativa de produzir a impressão nas condições válidas; herança nominal não prova conformidade comportamental. Testar cada implementação com entrada válida e verificar a saída esperada, além de garantir que o cliente comum usa somente o contrato. Não exigir que Python use declaração nominal de interface.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Arquivos e exceções (25 pontos)

```
def somar_arquivo(caminho):
    total = 0
    try:
        with open(caminho, encoding='utf-8') as arquivo:
            for numero, linha in enumerate(arquivo, 1):
                if not linha.strip():
                    continue
                try:
                    total += int(linha)
                except ValueError:
                    raise ValueError('inteiro inválido na linha ' + str(numero))
    except FileNotFoundError:
        raise FileNotFoundError('arquivo não encontrado: ' + str(caminho))
    return total
```

O contexto with fecha o arquivo nas saídas normais e excepcionais. A complexidade é linear na quantidade de texto; a memória adicional é constante desconsiderando a linha corrente. Solução Java com fechamento garantido e exceções correspondentes é aceitável.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Fluxos e limites de mensagens (35 pontos)

Fluxos não preservam as fronteiras das chamadas de envio. É necessário acumular bytes/texto até o delimitador.

```
buffer = vazio
para cada trecho recebido:
    buffer = buffer + trecho
    enquanto buffer contém nova_linha:
        linha, buffer = retirar_primeira_linha(buffer)
        analisar linha em comando e dois inteiros
        se válida: responder com a soma
        senão: responder erro de protocolo
no fechamento:
    se buffer não vazio: registrar mensagem incompleta
```

As respostas são 5 e 9. Leitor gerencia fluxo e fechamento; analisador valida quantidade de campos e números; calculador executa SOMA. Definir também limite de tamanho evita crescimento sem controle. Não presumir que uma leitura equivale a uma mensagem.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.